Week 3 Lesson 2 — Decision tree - Classification

8th November 2026

1 Decision tree - Classification

Article from Kaggle

We will use Decision tree classification to predict the result from our modal.

Lets import the libarary needed.

1.1 Dataset

The dataset is found on Kaggle

Context

Since as a beginner in machine learning it would be a great opportunity to try some techniques to predict the outcome of the drugs that might be accurate for the patient.

Content

The target feature is

- Drug type

The feature sets are:

- Age
- Sex
- Blood Pressure Levels (BP)
- Cholesterol Levels
- Na to Potassium Ration

Inspiration

The main problem here in not just the feature sets and target sets but also the approach that is taken in solving these types of problems as a beginner. So best of luck.

[257]:
```
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn import preprocessing
from sklearn.model_selection import train_test_split
from sklearn import metrics, tree

import matplotlib.pyplot as plt
```

```
from pathlib import Path
from urllib.request import urlretrieve

# Dataset
url = "https://huggingface.co/arminjodat/Drug_Classification/resolve/main/
 ↪drug200.csv"
file_path = Path("./data/week_3_drug200.csv")

file_path.parent.mkdir(parents=True, exist_ok=True)

if not file_path.exists():
    urlretrieve(url, file_path)
    print(f"Downloaded the dataset: {file_path}")
    size_bytes = file_path.stat().st_size
    print(f"Size: {size_bytes / 1024:.2f} KiB ({size_bytes / 1024**2:.2f} MiB)")
else:
    print(f"Dataset already exists: {file_path}")
```

```
Dataset already exists: data/week_3_drug200.csv
```

About the data: The data is about a list of pateint having a list of disease. Through the model we will predict which drug should be provided to the patient.

[258]:
```
df = pd.read_csv(file_path)
df.head()
```

[258]:

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	F	HIGH	HIGH	25.355	DrugY
1	47	M	LOW	HIGH	13.093	drugC
2	47	M	LOW	HIGH	10.114	drugC
3	28	F	NORMAL	HIGH	7.798	drugX
4	61	F	LOW	HIGH	18.043	DrugY

Lets Calculate the size of the data

[259]:
```
np.size(df)
```

[259]:
```
1200
```

[260]:
```
print(f"Shape of the data {df.shape[0]} rows and {df.shape[1]} columns")
```

```
Shape of the data 200 rows and 6 columns
```

Preparing the data

We will take some of the coloumns that are needed for the analaysis

[261]:
```
# The author works with series. But dataframes are better to keep the column␣
 ↪names
```

```
work_with_numpy_ndarray_bool = True

if work_with_numpy_ndarray_bool:
    X = df[['Age', 'Sex', 'BP', 'Cholesterol', 'Na_to_K']].values
else:
    # This returns a dataframe
    X = df[['Age', 'Sex', 'BP', 'Cholesterol', 'Na_to_K']].copy()
X[0:5]
```

[261]:
```
array([[23, 'F', 'HIGH', 'HIGH', 25.355],
       [47, 'M', 'LOW', 'HIGH', 13.093],
       [47, 'M', 'LOW', 'HIGH', 10.114],
       [28, 'F', 'NORMAL', 'HIGH', 7.798],
       [61, 'F', 'LOW', 'HIGH', 18.043]], dtype=object)
```

In the above array some values in the dataset are not numeric such as Sex or BP. Unfortunately, Sklearn Decision Trees cannot handle these variables. Hence we need to convert the same into numeric. `pandas.get_dummies()` Convert non-numeric variable into dummy/indicator variables.

[262]:
```
comparison = pd.DataFrame(index=df.index[:5])

le_sex = preprocessing.LabelEncoder()
le_sex.fit(['F','M'])

le_BP = preprocessing.LabelEncoder()
le_BP.fit([ 'LOW', 'NORMAL', 'HIGH'])

le_Chol = preprocessing.LabelEncoder()
le_Chol.fit([ 'NORMAL', 'HIGH'])

if work_with_numpy_ndarray_bool:
    X[:,1] = le_sex.transform(X[:,1])
    X[:,2] = le_BP.transform(X[:,2])
    X[:,3] = le_Chol.transform(X[:,3])

    for name, position in [('Sex', 1), ('BP', 2), ('Cholesterol', 3)]:
        comparison[f'df_{name}'] = df[name].iloc[:5]
        comparison[f'X_{name}'] = X[:5, position]
else:
    X["Sex"] = le_sex.transform(X["Sex"])
    X["BP"] = le_BP.transform(X["BP"])
    X["Cholesterol"] = le_Chol.transform(X["Cholesterol"])

    for name in ["Sex", "BP", "Cholesterol"]:
        comparison[f"df_{name}"] = df[name].iloc[:5]
        comparison[f"X_{name}"] = X[name].iloc[:5]

#X[0:5]
```

```
comparison
```

[262]:

	df_Sex	X_Sex	df_BP	X_BP	df_Cholesterol	X_Cholesterol
0	F	0	HIGH	0	HIGH	0
1	M	1	LOW	1	HIGH	0
2	M	1	LOW	1	HIGH	0
3	F	0	NORMAL	2	HIGH	0
4	F	0	LOW	1	HIGH	0

Above we have changed the values into 0,1,2 as numeric.

[263]:
```
#target value
y = df["Drug"]
y
```

[263]:
```
0      DrugY
1      drugC
2      drugC
3      drugX
4      DrugY
       …
195    drugC
196    drugC
197    drugX
198    drugX
199    drugX
Name: Drug, Length: 200, dtype: object
```

We will be using train/test split on our decision tree. Let's import train_test_split from sklearn.cross_validation.

[264]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,␣
↪random_state=3)
```

We wil create test and train data

We will first create an instance of the DecisionTreeClassifier called Tree.

[265]:
```
Tree = DecisionTreeClassifier(criterion="entropy", max_depth = 4)
Tree # it shows the default parameters
```

[265]:
```
DecisionTreeClassifier(criterion='entropy', max_depth=4)
```

Let's predict it

[266]:
```
Tree.fit(X_train,y_train)
```

[266]:
```
DecisionTreeClassifier(criterion='entropy', max_depth=4)
```

[267]:
```
pred =Tree.predict(X_test)
pred
```

[267]:
```
array(['DrugY', 'drugX', 'drugX', 'drugX', 'drugX', 'drugC', 'DrugY',
       'drugA', 'drugB', 'drugA', 'DrugY', 'drugA', 'DrugY', 'DrugY',
       'drugX', 'DrugY', 'drugX', 'drugX', 'drugB', 'drugX', 'drugX',
       'DrugY', 'DrugY', 'DrugY', 'drugX', 'drugB', 'DrugY', 'DrugY',
       'drugA', 'drugX', 'drugB', 'drugC', 'drugC', 'drugX', 'drugX',
       'drugC', 'DrugY', 'drugX', 'drugX', 'drugX', 'drugA', 'DrugY',
       'drugC', 'DrugY', 'drugA', 'DrugY', 'DrugY', 'DrugY', 'DrugY',
       'DrugY', 'drugB', 'drugX', 'DrugY', 'drugX', 'DrugY', 'DrugY',
       'drugA', 'drugX', 'DrugY', 'drugX'], dtype=object)
```

[268]:
```
print (pred [0:5])
print (y_test [0:5])
```

```
['DrugY' 'drugX' 'drugX' 'drugX' 'drugX']
40     DrugY
51     drugX
139    drugX
197    drugX
170    drugX
Name: Drug, dtype: object
```

[269]:
```
print("DecisionTrees's Accuracy: ", metrics.accuracy_score(y_test,pred))
```

```
DecisionTrees's Accuracy:  0.9833333333333333
```

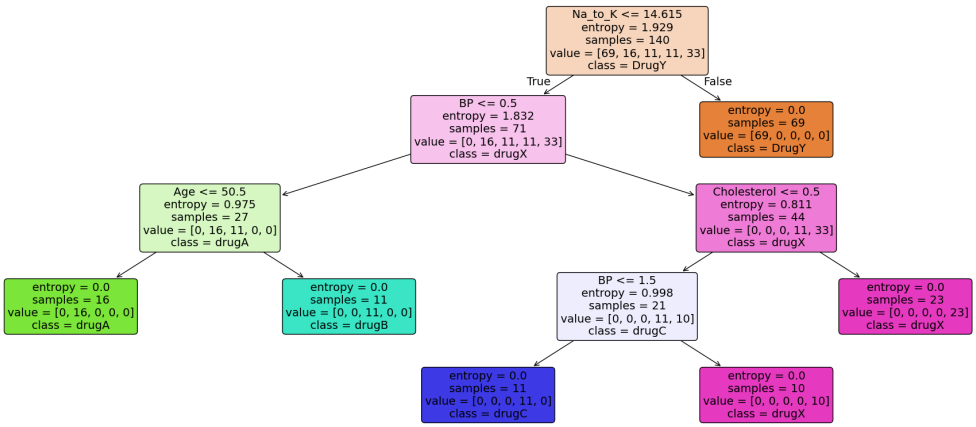
Create the Decision Tree and Visualize it!

[270]:
```
plt.figure(figsize=(25,10))
'''tree.plot_tree(Tree,filled=True,
                  rounded=True,
                  fontsize=14);'''

if work_with_numpy_ndarray_bool:
    # The same columns we extracted at the beginning of the notebook.
    feature_names = 'Age', 'Sex', 'BP', 'Cholesterol', 'Na_to_K'
else:
    feature_names=X.columns.tolist()

tree.plot_tree(
    Tree,
    feature_names=feature_names,
    class_names=[str(label) for label in Tree.classes_],
    filled=True,
    rounded=True,
    fontsize=14,
)
```

```
plt.show()
```



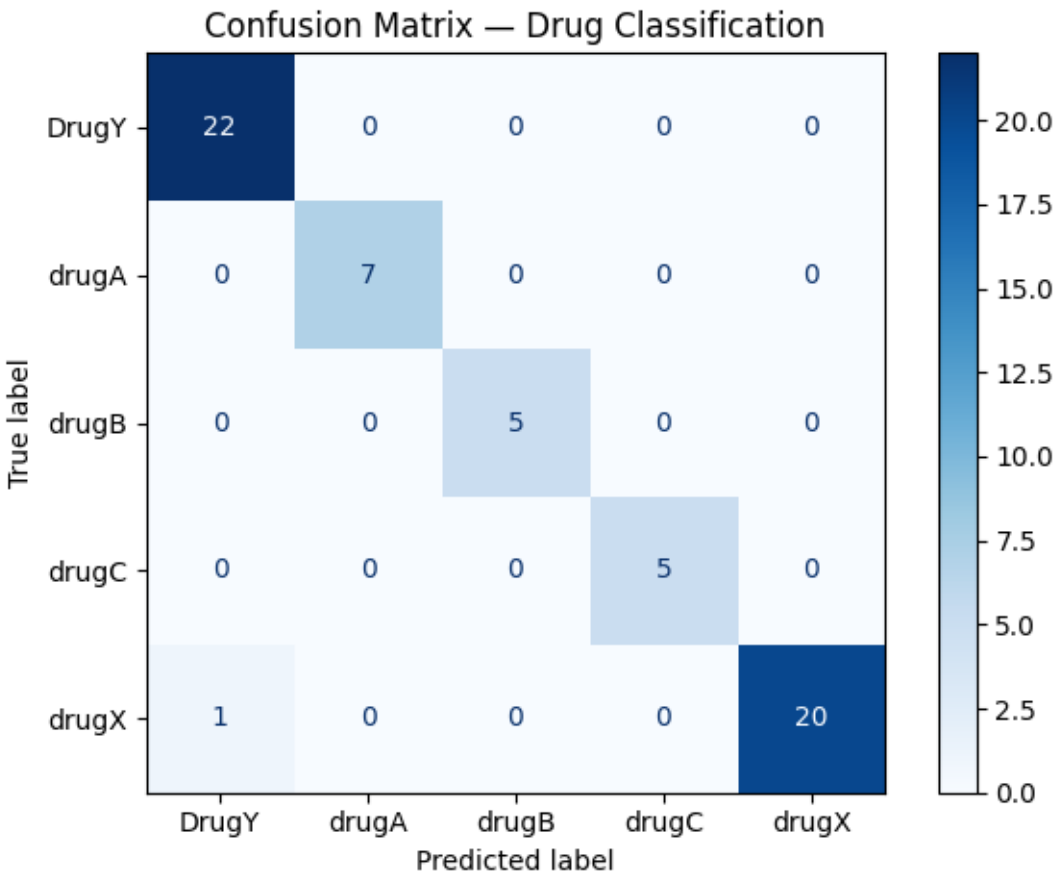
1.2 Extra: Confusion Matrix

[271]:
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Use the same drug class order for the matrix and its axis labels.
labels = Tree.classes_
cm = confusion_matrix(y_test, pred, labels=labels)

ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=labels,
).plot(cmap="Blues", values_format="d")

plt.title("Confusion Matrix - Drug Classification")
plt.tight_layout()
plt.show()
```


Confusion Matrix — Drug Classification
DrugY
drugA
drugB
drugC
drugX
True label
22 0 0 0 0
0 7 0 0 0
0 0 5 0 0
0 0 0 5 0
1 0 0 0 20
DrugY
drugA
drugB
drugC
drugX
Predicted label
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0